

Project of Software Correctness, Security and Reliability

Checkers

A checker is a static analysis component that, starting from the results produced by the abstract analysis of a program, verifies whether specific properties of interest are satisfied (or potentially satisfied) at certain points in the Control Flow Graph.

In LiSA, a checker does not execute the analysis itself but it queries the analysis. For each node of the CFG of interest, it extracts the post-execution abstract state of that instruction and checks whether the abstract value of the involved expressions satisfies a problematic condition. If so, it issues a warning. There are two types of warnings:

Definite: The abstract domain guarantees that the property is violated in all concrete cases (the denominator is exactly $[0,0]$ using a interval abstract domain).

Possible: The abstract domain cannot assure that the property is violated, but does not guarantee that the property is satisfied (the denominator is $[-1, 1]$, which includes zero but is not precisely zero).

Div By Zero Checker

This checker checks division operations within the CFG. For each division node, it extracts the abstract value of the denominator using the `Extended_IntervalLattice` domain (in the case of our analysis). It issues a definite warning if the interval is exactly $[0,0]$, and a possible warning if the interval includes 0 (like in the case of $[-1, 1]$).

Taint Three Levels Checker

It checks calls to functions annotated as `@sink`. For each parameter passed to a sink, it extracts the abstract value within the `TaintThreeLevelsLattice` domain (which has the levels Bottom, Clean, Tainted, Top). It issues a definite warning if the value is `TAINTED`, and a possible warning if it is `Top` (it is unknown whether it is tainted or clean).

String Checker

HTTPStringChecker: It checks function calls and inspects the passed parameters. It uses the `StrPrefix` domain to extract the abstract prefix of each string argument. It issues a warning if the prefix starts with `"http://"` or `"https://"`.

DotComStringChecker: Similar to the previous, but it uses the `StrSuffix` domain. It checks the parameters of function calls and warns if the abstract suffix of an argument ends with `.com`.

Overflow/Underflow Checker

This checker checks arithmetic operations (like addition, subtraction, multiplication, division). It is made with a range of integers in a form like this: $[l, u]$ (for example, the limits of a 32-bit int). After each operation, it verifies whether the abstract result exceeds that range: it issues an overflow warning if it exceeds the upper bound, and an underflow warning if it falls below the lower bound.

Analysis Procedure

The procedure adopted to analyze ten input programs for each checker was divided into several phases.

1. The first phase consisted of selecting the input programs to be analyzed with the checker, ensuring they were sufficiently different and not too similar.
2. The second phase was the configuration/setting phase. For each checker, it was necessary to ensure the presence of the test file and that this test file implemented the correct abstract domain (extended interval in our case) and added the proper checker to be used.
3. Then the analysis was launched and the results were collected.

In particular, the file “report.json” was consulted to see the warnings produced by the checker in use. Then, the source code of the input file was taken and both were compared with the CFG and the checker's code to verify that the results were consistent. This comparison produced a series of false positives and/or false negatives, allowing for conclusions and considerations about the checker's behavior.

To facilitate the reading of some particular cases, their results from the application of the checkers are reported below but only the warnings requiring particular attention are shown. To view the complete list of warnings for each file, there is a dedicated section at the end of the document.

Div By Zero Checker

Abstract Domain: Extended_Interval

Checker: DivByZeroIntervalChecker.java

Test Files: NumericalAnalysesTest.java

Input Files:

- 876957-div_by_zero.imp

"warnings" : ["157:28 [This may be possible division by zero]", "203:37 [This may be possible division by zero]"]

```
152 class SettlementEngine {
153
154     splitAmount(total, participants) {
155         def share = 0;
156         if (participants > 0) {
157             share = total / participants;
158         }
159         return share;
160     }
161
162     main() {
163         def total = 50000;
164         def participants = 4;
165         def result = this.splitAmount(total, participants);
166         return result;
167     }
168 }
```

```
198 class RequestDispatcher {
199
200     computeShare(total_requests, workers) {
201         def share = 0;
202         if (workers > 0) {
203             share = total_requests / workers;
204         }
205         return share;
206     }
207
208     main() {
209         def requests = 320;
210         def w = 4;
211         def result = this.computeShare(requests, w);
212         return result;
213     }
214 }
```

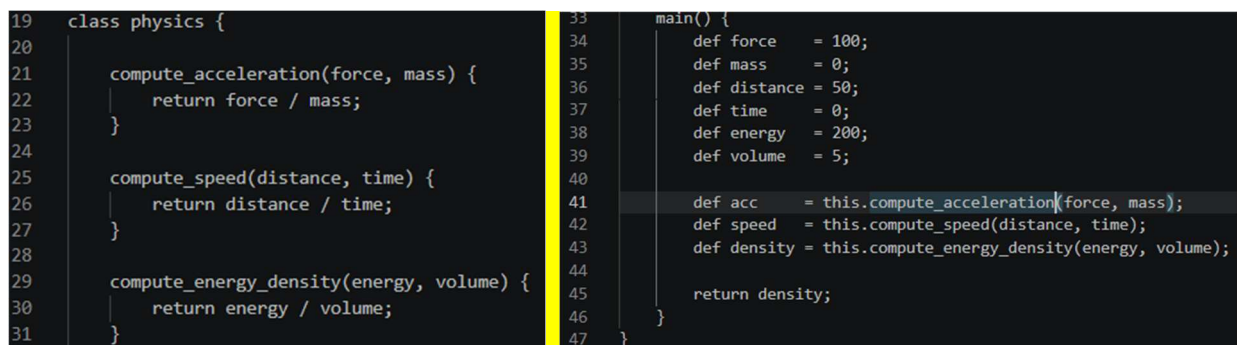
The analysis of this file shows the trade-off between soundness and precision in the static analysis framework Lisa.

In the “splitAmount” function (line 157) for example, it can be observed that the division by the “participants” variable is guarded by a conditional guard: “if (participants > 0)”. At runtime, this check guarantees that a division by zero is impossible. However, the static analysis propagates a warning because it operates in a context-insensitive way at the inter-procedural level: the formal input parameters of the function are initialized as the Top element of the abstract lattice (the interval $[-\text{inf}, +\text{inf}]$ since we are using the `Extended_Interval` as the abstract domain).

The approximation introduced by the lack of context-sensitivity plus the way Lisa models branch conditions prevents the `Extended_Interval` abstract domain from completely excluding zero from the post-condition abstract state. The checker is forced to emit a possible warning so to preserve soundness and avoid omitting false negatives.

- 881299_divbyzero.imp

```
"warnings" : [  
  "22:23 [This may be possible division by zero]",  
  "26:26 [This may be possible division by zero]",  
  "30:24 [This may be possible division by zero]" ]
```



```
19 class physics {  
20  
21     compute_acceleration(force, mass) {  
22         return force / mass;  
23     }  
24  
25     compute_speed(distance, time) {  
26         return distance / time;  
27     }  
28  
29     compute_energy_density(energy, volume) {  
30         return energy / volume;  
31     }  
32 }  
  
33 main() {  
34     def force = 100;  
35     def mass = 0;  
36     def distance = 50;  
37     def time = 0;  
38     def energy = 200;  
39     def volume = 5;  
40  
41     def acc = this.compute_acceleration(force, mass);  
42     def speed = this.compute_speed(distance, time);  
43     def density = this.compute_energy_density(energy, volume);  
44  
45     return density;  
46 }  
47 }
```

Here in the `main()` method, the program invokes the functions of the “physics class” by passing constants with value zero (such as `mass = 0` at line 22 and `time = 0` at line 26), which create problems at runtime. The checker flags these lines as possible warnings rather than definite warnings and a similar warning is raised at line 30, where the passed argument is a safe value (`volume = 5`).

This symmetrical behavior proves that the abstract domain isolates the analysis of individual procedures from the invocation context of their callers. Since the divisors are variables passed as arguments, the interval domain preemptively assigns them the abstract state $[-\text{inf}, +\text{inf}]$. Consequently, because it contains zero, any division operation featuring a generic parameter in the denominator systematically generates a possible warning. The framework deliberately chooses to accept a loss of precision (generating false positives, as seen at line 30) to preserve the soundness of the analysis.

- **896827-895879-divbyzero.imp**

"warnings" : ["18:23 [This may be possible division by zero]", "39:34 [This may be possible division by zero]"]

<pre> 10 // CASE 2: divisor shrinks toward zero inside a loop 11 shrinkingDenominator() { 12 def value = 1000; 13 def denom = 80; 14 while (denom > 0) { 15 denom = denom - 3; 16 } 17 18 value = value / denom; 19 return value; 20 } </pre>	<pre> 32 // CASE 4: unguarded division by external input (TOP interval) 33 guardedDivision(divisor) { 34 def dividend = 100; 35 36 def unsafe = dividend / divisor; 37 38 if (divisor > 0) { 39 def safe = dividend / divisor; 40 return safe; 41 } 42 return unsafe; 43 } </pre>
--	--

This test case shows the loss of precision induced by the analysis of iterative constructs (loops).

At line 15, in the while loop body, a variable is decremented at each iteration ($\text{denom} = \text{denom} - 3$), starting from an initial value of 80. The execution will terminate when “denom” reaches the value of -1 as a result, the operation at line 18 will never encounter zero at runtime in the denominator. Yet, the checker raises a possible warning.

This happens because the static analysis of loops requires approximating the fixed point of the abstract transfer function. To guarantee convergence and termination of the algorithm within a finite time (Ascending Chain Condition), the widening operator is applied. Within the interval domain, widening tends to collapse unstable boundaries toward infinity. In this control flow, the conservative approximation causes the abstract state computed for denom at the exit of the iterations to stabilize at the interval $[-\infty, 0]$. Since zero is included as the upper bound of the abstract interval, the checker triggers the warning for a possible division by zero. .

For the warnings issued at lines 36 and 39, the scenario is identical to the one described in the test case description of the previous file “876957-div_by_zero.imp” (path-insensitivity of the analysis over conditional control structures).

- **divbyzero.imp**

"37:20 [This may be possible division by zero]", "45:25 [This may be possible division by zero]", "89:16 [This may be possible division by zero]"]

<pre> 27 function_call_branching(){ 28 def i = this.get_zero(); 29 def x = 4; 30 if(i > 0){ 31 x = 1; 32 } 33 else{ 34 x = 0; 35 } 36 37 def err = 1/x; 38 } </pre>	<pre> 40 // Array access 41 array_access(){ 42 def arr = new int[1]; 43 arr[0] = 0; 44 45 def err = 1/arr[0]; 46 } </pre>	<pre> 81 looping_array(){ 82 def err = 0; 83 def arr = new int[1]; 84 arr[0] = 0; 85 for (def i = 5; i >= 0; i = i - 1){ 86 arr[0] = i; 87 } 88 def denom = arr[0]; 89 err = 1/denom; 90 } </pre>
--	---	--

In the line 37 the checker issues a “possible” division by zero caused by the join of the two condition branches (where $x = 1$ or $x = 0$ depending on the value of the variable “i”). However after the source code it was clear that the `get_zero()` function just returns 0. This implies that the if-guard will always evaluate to false, and the program will only execute the else branch, assigning $x = 0$. The instruction $1 / x$ therefore represents a certain division by zero. However, the checker classifies it only as a possible warning.

This happens because of the behavior of the join operator (specifically, the LUB) at flow. In the program's CFG, the if branch (where the abstract state is x in $[1, 1]$) and the else branch (where the abstract state is x in $[0, 0]$) reconverge right before the division instruction. The join operator merges the two intervals, computing a combined abstract interval where x is in $[0, 1]$.

Since the resulting interval $[0, 1]$ contains zero but is not restricted to the point $[0, 0]$, the checker is not secure about the violation (as it cannot statically determine which branch will actually be taken without an appropriate dead-code analysis) and downgrades the warning from definite to possible.

Taint Three Levels Checker

Abstract Domain: TaintThreeLevelsSolution.java

Checker: TaintThreeLevelsCheckerSolution.java

Test File: TaintThreeLevelsAnalysisTest.java

Input Files:

- 872966_TaintThreeLevels.imp

"warnings" : ["104:37 [There is a possible tainted value in a sink]", "78:43 [There is a possible tainted value in a sink]", "86:38 [There is a possible tainted value in a sink]"]

```
73 // Branch logic (bypass vulnerability)
74 if (bypassShield == true) {
75     // VULNERABILITY: Tainted data reaches sink without sanitization
76     // Logic error: assuming priorityLevel 10 is always safe
77     if (priorityLevel > 10) {
78         this.db_execute(transformed); // <--- WARNING EXPECTED
79     }
80 } else {
81     // 5. Multi-Stage Sanitization
82     def level2 = this.basicSanitize(transformed);
83
84     // Check if checker recognizes 'level2' as now SAFE
85     if (priorityLevel < 5) {
86         this.db_execute(level2); // <--- SHOULD BE SAFE
87     } else {
88         // 6. Escalation to level 3 (Encryption)
89         def level3 = this.advancedEncrypt(level2);
90         this.db_execute(level3); // <--- DEFINITELY SAFE
91     }
92 }
93 }
```

```
95 // 7. Data Leak via Inter-Object Transfer
96 testDataLeak() {
97     def secret = this.getInternalToken();
98     def publicData = this.getExternalRequest();
99
100     // Mixing Clean and Tainted data
101     def mixed = secret + " | " + publicData;
102
103     // Passing to a sink
104     this.log_to_public_file(mixed); // <--- WARNING EXPECTED:
105 }
106 }
```

The analysis of this test case highlights the conservative behavior of the abstract domain when evaluating compound expressions and conditional flows. At line 104, the variable `publicData` is defined as Tainted since it is defined using a function that in the documentation is described as returning a taint value. However, during the abstract evaluation of the assignment to the variable “mixed” (which inherits its value in the line 101), the post-condition abstract state fails to maintain precision and degrades to Top. By checking the join node in the control flow graph, it was clear that the abstract transfer function propagates the Top element of the lattice instead of the Tainted. This downgrading is a decision to preserve structural soundness at the expense of local precision. A similar loss of precision occurs at lines 86 and 90. Despite invoking specific sanitization functions (which should theoretically force the abstract state to Clean), the CFG is not modeled to capture the specific semantics of the sanitizer leaving the abstract environment Top leading to a false positive.

- **880119-890558-taint-three-levels-2.imp**

"warnings" : ["26:38 [There is a possible tainted value in a sink]", "34:37 [There is a possible tainted value in a sink]", "58:36 [There is a possible tainted value in a sink]"]

```

24  system_input_non_sanitized(req) {
25      def input = this.source1();
26      def result = this.system(input); // taint
27      return result;
28  }

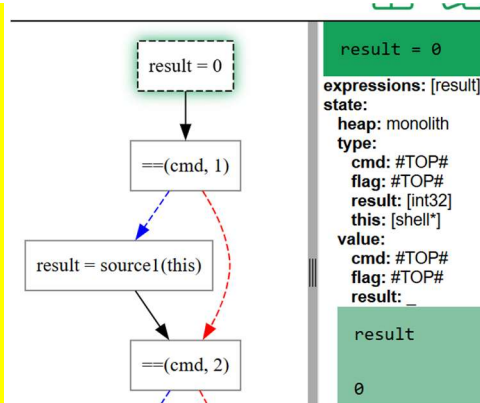
30  system_input_sanitized_completely() {
31      def data = this.source1();
32      data = this.strip1(data);
33      data = this.strip2(data);
34      def result = this.system(data); // Clean
35      return result;
36  }

```

```

47  run_app(cmd, flag) {
48      def result = 0;
49
50      if (cmd == 1) {
51          result = this.source1();
52      }
53      if (cmd == 2) {
54          def data = this.source1();
55          if (flag > 0) {
56              data = this.strip1(data);
57          }
58          result = this.system(data); // Possibly tainted
59      }
60
61      if (cmd == 3) {
62          result = this.system(this.source1()); // Tainted
63      }
64
65      return result;
66  }

```



This test case highlights the limitations imposed by inter-procedural context-insensitivity in Lisa. At line 26, there is a flow from an untrusted source to a system sink but the checker only issues a potential warning (possible tainted). The underlying cause resides in the initialization of the parameters, lacking any knowledge about the current invocation context, the framework assigns Top to the input arguments. This over-approximate the potential threat and prevents the emission of a warning “definitely tainted”. At line 58, we observe a flaw in the initial abstraction function. The variable “result” is initialized with a literal constant, which should be Clean. However, an inspection of the CFG revealed an unexpected contamination of the environment, where the constant is promoted to Top.

- **894004_taint.imp**

"warnings" : ["117:30 [There is a possible tainted value in a sink]", "129:30 [There is a possible tainted value in a sink]"]

```

115  unsafeDirectQuery() {
116      def userId = this.GetRequest();
117      this.runQueryDB(userId);
118  }

126  sanitizedQuery() {
127      def userId = this.GetRequest();
128      def safeId = this.sanitizer1(userId);
129      this.runQueryDB(safeId);
130  }

```

This scenario allows us to see one again the checker issues a warning of “possible tainted” at line 117 when the function used (runQueryDB) was previously documented as returning a tainted value. At line 128 it is possible to see the lack of sanitization of the “userId” variable that lead to a warning when it is used in the next line (line129).

- **1003406_three_taint_1.imp**

"warnings" : ["78:37 [There is a possible tainted value in a sink]", "98:37 [There is a possible tainted value in a sink]"]

```

75  runReportUnsafe() {
76      def header = this.getExternalRequest();
77      def cmd = "convert " + header;
78      this.executeSystemCommand(cmd);
79  }

94  runReportSafe() {
95      def raw = this.getExternalRequest();
96      def safe = this.sanitizeParam(raw);
97      def cmd = "convert " + safe;
98      this.executeSystemCommand(cmd);
99  }

```

This scenario allows us to see one again the checker issues a warning of “possible tainted” at line 78 when the function used at line 76 (which provides the value of the variable “header” used in the definition of the variable “cmd” used in the line of the warning) was previously documented as returning a tainted value. At line 96 it is possible to see the lack of sanitization of the “raw” variable that lead to a warning when it is used in the line 98.

String Checker

Abstract Domain: Prefix, Suffix

Checker: HTTPStringChecker, DotComStringChecker

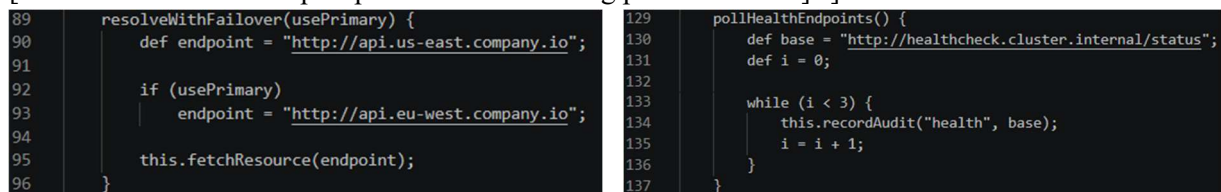
Test Files: StringAnalysesTest.java

Input Files:

- 872966_Strings.imp

HTTPStringChecker

"warnings" : ["134:43 [The function uses an http/https url in the following parameters: 2nd]", "95:35 [The function uses an http/https url in the following parameters: 1st]"]



```

89     resolveWithFailover(usePrimary) {
90         def endpoint = "http://api.us-east.company.io";
91
92         if (usePrimary)
93             endpoint = "http://api.eu-west.company.io";
94
95         this.fetchResource(endpoint);
96     }
129     pollHealthEndpoints() {
130         def base = "http://healthcheck.cluster.internal/status";
131         def i = 0;
132
133         while (i < 3) {
134             this.recordAudit("health", base);
135             i = i + 1;
136         }
137     }

```

The analysis demonstrates that the checker does not perform a simple scanning of string definitions. The warning at line 95 is generated by extracting the post-execution abstract state immediately preceding the invocation, verifying if the abstract value satisfies the violation condition (the prefix element contains or matches the http:// or https://).

At line 134, an invocation is intercepted where the argument contains “http://” even though the function “recordAudit” has an empty body. This behavior proves that the checker implements a control logic based on call semantics. The checker queries the abstract state of the caller without needing to analyze the caller's body preserving the soundness.

- 880119-890558-dotcom-strings.imp

HTTPStringChecker

"warnings" : []

DotComStringChecker

"warnings" : [{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp:113:32] on 'untyped DotComStrings::cdnRouter(DotComStrings* this, untyped region)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':136:53] on 'untyped DotComStrings::phishingEmailDetection(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

},{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':172:37] on 'untyped DotComStrings::dnsRebindingSetup(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

},{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':24:31] on 'untyped DotComStrings::directDotCom(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

},{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':32:41] on 'untyped DotComStrings::directDotComEmail(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

},{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':54:43] on 'untyped DotComStrings::buildEmailAddress(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

},{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':70:47] on 'untyped DotComStrings::corporateEmailSystem(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

},{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':92:41] on 'untyped DotComStrings::serverSelection(DotComStrings* this, untyped useBackup)': [EXPRESSION] The function uses a .com in the following parameters: 1st"}]

- 894579_896954_prefixsuffix_5.imp

HTTPStringChecker

"warnings" : ["19:40 [The function uses an http/https url in the following parameters: 1st]"]

DotComStringChecker

"warnings" : ["19:40 [The function uses a .com in the following parameters: 1st]"]

```

17     testUpgrade() {
18         def raw = "http://google.com";
19         def upgraded = this.upgrade(raw);
20         return this.send(upgraded);
21     }

```

The asymmetric behavior observed between lines 19 and 20 shows the flow-sensitive nature of the analysis and the “destruction” of abstract information caused by the transfer functions of string domains.

The “raw” variable is passed as an argument first to the upgrade function (line 19, where the warning is raised) and immediately after to the send function (line 20, where no warning is emitted).

- 896827-895879-prefix.imp

HTTPStringChecker

"warnings" : ["89:67 [The function uses an http/https url in the following parameters: 1st]"]

```

33     // CASE 4: join of two branches (one with http)
34     joinedOneHttpRequest(flag) {
35         def target = "http://primary.com/home";
36         if (flag) {
37             target = "foo";
38         }
39         this.redirect(target);
40     }

```

```

86     generateReportVulnerable() {
87         //template_name should be passed as parameter to the function, f
88         def template_name = "http://internal-service.corp/admin/dump";
89         def safe_name = this.sanitizeTemplateName(template_name);
90         this.renderTemplate(safe_name);
91     }

```

This test case illustrates the impact of path-insensitivity and the handling of conditional branches of relational numeric constraints in the LiSA.

At line 39, where the conditional usage of a string who could contain “http://” depends on the evaluation of a boolean variable “flag”, whose value is unknown Top but no warnings were issued.

When the control flow encounters an if-else statement, Lisa independently analyzes the then and else branches. Since the abstract “assume” operator lacks of instructions capable of linking the logical value of “flag” variable to the state of the “target” string, the computations of the two paths diverge. At the convergence point of the flow (the join point in the CFG), the LUB operator is applied to merge the abstract states. If the variable remains safe or unmodified in one branch but contains the “http://” prefix in the other, the LUB between a safe string and a known prefix yields an approximation. Because the checker requires sufficient precision to determine the existence of the prefix along all joining execution paths, the potential non-execution of the if body due to the unknown value of “flag” leads the framework to approximate the prefix property to Top.

The warning issued at line 89 initially lead to think that the sanitizeTemplateName function is missbehaving and it is not sanitizing correctly the string, but after a more careful analysis of the source code, it was clear that such function was modelled as a broken sanitize function.

Overflow/Underflow Checker

Abstract Domain: Extended_Interval

Checker: OverflowIntervalChecker

Test File: NumericalAnalysesTest.java

Input Files:

- 876957-overflow.imp

OVERFLOW 8-BITS

"warnings" : ["155:42 [This is an over/underflow]"]

OVERFLOW 16-BITS

"warnings" : ["147:42 [This is an overflow]", "155:42 [This is an over/underflow]", "155:42 [This is an underflow]"]

OVERFLOW 32BITS

"warnings" : ["147:42 [This is an over/underflow]", "147:42 [This is an overflow]", "155:42 [This is an over/underflow]", "155:42 [This is an underflow]"]

```
143     deposit(amount) {
144         if (amount > 0) {
145             // REALISTIC BUG: depositing a huge amount into an already large account
146             // can overflow the 32-bit signed integer limit, making balance negative.
147             this.balance = this.balance + amount;
148         }
149     }
150
151     withdraw(amount) {
152         if (amount > 0) {
153             // REALISTIC BUG: withdrawing a huge amount from an already very negative
154             // account can underflow, wrapping the 32-bit integer to a large positive.
155             this.balance = this.balance - amount;
156         }
157     }
```

After many tests, it was clear that the checker mapped warnings regarding potential integer boundary overflows directly in the internal blocks of the functions, rather than at the callers' invocation points. This behavior is caused by the context-insensitive of the inter-procedural analysis implemented by Lisa. When a function is statically analyzed in isolation, its parameters do not inherit the bounded intervals of the actual parameters defined in the main(). They are conservatively initialized to Top, representing the interval

$[-\text{inf}, +\text{inf}]$. Consequently, the application of abstract transfer functions for addition or multiplication operators over unbounded ranges triggers the expansion of the post-condition interval. To preserve the soundness of the verification, the checker issues a possible warning.

- 880119-890558-overflow.imp

OVERFLOW 8-BITS

"warnings" : ["46:25 [This is an overflow]", "50:25 [This is an overflow]"]

OVERFLOW 16-BITS

"warnings" : ["46:25 [This is an overflow]", "50:25 [This is an overflow]"]

OVERFLOW 32BITS

"warnings" : ["46:25 [This is an overflow]", "50:25 [This is an overflow]"]

<pre> 3 f0() { 4 def a = 2147483640; 5 def b = 1000; 6 def c = a + b; 7 def d = b + b; 8 9 return; 10 }</pre>	<pre> 39 f4() { 40 def a = this.readInt(); 41 def result = 0; 42 43 if (a > 0) { 44 def x = 2147483640; 45 def y = 100; 46 result = x + y; 47 } else { 48 def x = 100; 49 def y = 200; 50 result = x + y; 51 } 52 53 return; 54 }</pre>
---	--

This test scenario illustrates the flow-sensitive property of the analysis over the Control Flow Graph and the progressive degradation of abstract information. In the initial assignment lines 4 and 5, variables are modeled using literal constants or operations whose interval bounds are statically sound and strictly contained within the physical thresholds of the selected bit-width architecture. At this stage, the abstract state is highly precise, resulting in zero warnings. However, as the analysis traverses the program's definition-use chains (def-use chains) toward subsequent operations (lines 6, 7, 46, 50), the variables undergo sequential transformations or encounter control flow convergence points (join nodes). At these points, the Least Upper Bound (LUB) operator merges intervals originating from alternative execution paths. If this over-approximation introduces an interval expansion that reaches or overshoots the boundaries $[l, u]$, the abstract state suffers an irreversible loss of precision. When the checker queries the post-transfer function state at downstream nodes, it detects this potential boundary violation and appropriately raises warnings, demonstrating how abstract imprecision propagates transitively across subsequent computational blocks.

- **894579_896954_overunderflow_1-2.imp**

OVERFLOW 8-BITS

"warnings" : ["20:35 [This is an over/underflow]", "20:44 [This is an over/underflow]", "30:43 [This is an over/underflow]", "40:45 [This is an over/underflow]"]

OVERFLOW 16-BITS

"warnings" : ["20:35 [This is an over/underflow]", "20:44 [This is an over/underflow]", "30:37 [This is an over/underflow]", "30:43 [This is an over/underflow]"]

OVERFLOW 32BITS

"warnings" : ["20:35 [This is an over/underflow]", "20:44 [This is an over/underflow]", "25:45 [This is an over/underflow]", "30:37 [This is an over/underflow]", "30:43 [This is an over/underflow]", "35:45 [This is an over/underflow]"]

<pre> 8 class FsContext { 9 pageSize; 10 bufferSize; 11 legacyData; 12 13 ~FsContext() { 14 this.pageSize = 4096; 15 this.bufferSize = 0; 16 this.legacyData = ""; 17 } </pre>	<pre> 19 appendParam(data, len) { 20 def tmp = this.pageSize - 2 - this.bufferSize; 21 if (len > tmp) { 22 return -1; 23 } 24 this.copy(data, len); 25 this.bufferSize = this.bufferSize + len; 26 return 0; 27 } </pre>
--	---

```

29     appendParamFixed(data, len) {
30         def tmp = this.bufferSize + len + 2;
31         if (tmp > this.pageSize) {
32             return -1;
33         }
34         this.copy(data, len);
35         this.bufferSize = this.bufferSize + len;
36         return 0;
37     }
38
39     copy(data, len) { // Simulated unsafe memory copy
40         this.legacyData = this.legacyData + data;
41     }
42
43     corruptState() { // simulates attacker-controlled corruption
44         this.bufferSize = 5000;
45     }
46
47     copy(data, len) { // Simulated unsafe memory copy
48         this.legacyData = this.legacyData + data;
49     }

```

The occurrence of multiple distinct warnings tied to the exact same line of source code (e.g., lines 20 and 30) reflects how the checker inspects the CFG. A line of code that comprises compound arithmetic operations (for example $(a + b) * c$) is decomposed by Lisa into a sequence of distinct intermediate nodes within the CFG, and the checker queries the abstract post-state of each individual operator node. Consequently, if both the operands exceed the interval limits, separate and independent warnings are generated. Regarding lines 34 and 35, it is important to highlight that unknown parameters fail to immediately trigger warnings.

- overflow8.imp

OVERFLOW 8-BITS

"warnings" : ["15:38 [This is an overflow]", "17:30 [This is an overflow]", "20:30 [This is an overflow]", "29:38 [This is an overflow]", "31:30 [This is an overflow]", "34:30 [This is an overflow]", "43:29 [This is an overflow]", "56:36 [This is an over/underflow]", "5:22 [This is an overflow]", "7:22 [This is an underflow]"]

OVERFLOW 16-BITS

"warnings" : ["15:38 [This is an overflow]", "17:30 [This is an overflow]", "20:30 [This is an overflow]", "29:38 [This is an overflow]", "31:30 [This is an overflow]", "34:30 [This is an overflow]"]

OVERFLOW 32BITS

"warnings" : ["15:38 [This is an overflow]", "17:30 [This is an overflow]", "20:30 [This is an overflow]", "29:38 [This is an overflow]", "31:30 [This is an overflow]", "34:30 [This is an overflow]"]

<pre> 11 byte_loop_branching_overflow(){ 12 def max = 10; 13 def byte = 125; 14 def flag = 1; 15 for(def i = 0; i < max; i = i+1){ 16 if(flag == 1){ 17 byte = byte + 1; 18 } 19 else{ 20 byte = byte + 0; 21 } 22 } 23 } </pre>	<pre> 46 get_one(){ 47 return 1; 48 } 49 get_zero(){ 50 return 0; 51 } 52 byte_function_call(){ 53 def byte = 127; 54 byte = byte + this.get_one(); 55 byte = 0; 56 byte = byte + this.get_zero(); 57 } </pre>
---	--

This test case shows a false positive at line 56 (`get_zero()`), in fact the variable “byte” is equal to 127 (max of 8bits integers) and the function `get_zero()` returns always zero so no warning should be issued but due to the absence of an inter-procedural analysis, Lisa handles the assignment of the function call conservatively, initializing the destination variable to $[-\infty, +\infty]$ in the 8-bits interval. In 16-bit and 32-bit scenarios, the broader tolerance margins could prevent an immediate warning.

Regarding the lines 15, 17, 20, 29, 31 and 34, in iterative constructs (like while loops), static analysis must compute the fixed point of the abstract transfer function sequence. Because the interval lattice contains infinite ascending chains (Ascending Chain Condition not satisfied), the framework is forced to apply a Widening operator to guarantee algorithm termination within finite time. The widening operator extrapolates the boundaries of intervals that increase or decrease between successive iterations, leading to $+\infty$ or $-\infty$. This over-approximation goes in conflict with local precision. As control flow advances, it becomes impossible to exclude overflows/underflows. So the checker has to emit a warning to preserve the soundness.

Implicit Checker

The checker analyzes implicit flows, situations where tainted information propagates not through direct assignments, but through the program's control flow.

```
def secret = this.source1(); // TAINTED
def result = 0;              // CLEAN

if (secret > 0) {
    result = 1;              // result
}
```

It works as follows:

- Check the condition: For each node representing a condition (if, while), it verifies whether the variables contained in it are tainted. If at least one is, it issues a warning on the condition itself.
- Check the body: If the condition is tainted, it inspects all statements inside the branch/loop. For every variable found, it checks whether it was clean before the condition and if it was, it issues a warning because that variable might have been modified based on a tainted value.

A base taint analysis checker (like `TaintThreeLevelsCheckerSolution` used in the section dedicated to the test cases with the `TaintThreeLevelsChecker`) would not issue a warning on “result” (see image above) because it is never directly assigned a tainted value. However, result leaks information about secret through the control flow, if `result == 1` the attacker deduces that `secret > 0`. Or simpler, a clean variable “result” is manipulated in an if body that depends on a tainted (not trusted) variable.

In the implicit flow, the taint propagates through the control structure, not through data. The base checker (`TaintThreeLevelsChecker`) only covers explicit flows.

The checker I developed is not a standalone checker, it only detects implicit flows, so in order to achieve a complete taint analysis it should be used together with a base taint analysis checker for the explicit flows detection.

In a first test, I used my checker alone and the resulting warnings are the following on the next page. And the checker was able to detect all the tainted conditions in loops/if-else structures but it wasn't able to detect explicit flows as expected (line 61 and line 147):

```

40      def t = this.source1();
41      def c = 0;
42
43      while(t<3)
44      {
45          c = c + 1;
46          t = t + 1;
47      }
48
49      c = 6;
50
51      while(t<3)
52      {
53          c = c-1;
54          t = c;
55      }
56
57      c = this.source1();
58
59      def d = c;
60
61      this.sink1(c);
62  }

```

```

130      f5()
131      {
132          def t = this.source1();
133          def c = 0;
134
135          if(c==0 || t>2)
136          {
137              c = c + 1;
138              t = t + 1;
139          }
140
141          if(c==2)
142          {
143              c = this.source1();
144              t = t + 1;
145          }
146
147          this.sink1(c);
148      }

```

(As expected the warnings at lines 43, 45, 51, 53, 135, 137 were issued)

In a second test, i used my checker together with Base Three Taint Analysis Checker and both the missing warnings from the previous test were covered. In the source code will be other two input programs AI-generated with their versions and the prompts used to create such tests.

First Test

Abstract Domain: TaintThreeLevels

Checker: ImplicitFlow_Checker

Test File: ImplicitFlow_CheckerTest

Input Files: implicitflow_checker.imp

"warnings" : [{

 "message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':115:17] on 'untyped hc::f4(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

 "message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':117:24] on 'untyped hc::f4(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':117:24, variable modified inside a conditional statement, it should be considered tainted"

}, {

 "message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':136:21] on 'untyped hc::f5(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

 "message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':138:20] on 'untyped hc::f5(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':138:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':142:19] on 'untyped hc::f5(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':144:20] on 'untyped hc::f5(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':144:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':160:21] on 'untyped hc::f6(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':162:25] on 'untyped hc::f6(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':164:24] on 'untyped hc::f6(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':164:24, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':184:30] on 'untyped hc::f7(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':186:20] on 'untyped hc::f7(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':186:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':24:13] on 'untyped hc::f0(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':25:20] on 'untyped hc::f0(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':25:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':43:16] on 'untyped hc::f1(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':45:20] on 'untyped
hc::f1(hc* this): [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':45:20, variable modified inside a
conditional statement, it should be considered tainted"
```

```
}, {
```

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':51:16] on 'untyped
hc::f1(hc* this): [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
```

```
}, {
```

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':53:18] on 'untyped
hc::f1(hc* this): [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':53:18, variable modified inside a
conditional statement, it should be considered tainted"
```

```
}, {
```

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':75:11] on 'untyped
hc::f2(hc* this): [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
```

```
}, {
```

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':77:20] on 'untyped
hc::f2(hc* this): [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':77:20, variable modified inside a
conditional statement, it should be considered tainted"
```

```
}, {
```

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':94:21] on 'untyped
hc::f3(hc* this): [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
```

```
}, {
```

```
"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':96:20] on 'untyped
hc::f3(hc* this): [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':96:20, variable modified inside a
conditional statement, it should be considered tainted"
```

```
} ]
```

After these two test, i evaluated the performance of the implicitflow_checker by generating two input files with different AI to see if the checker kept performing well in each situation. About the file “implicitflow_checker3.imp” generated by Gemini AI, all the warnings were issued correctly, this slightly changed with the file “implicitflow_checker2.imp” generated by Claude AI, in which all warnings but one were issued correctly (see line 131)

```
test_sessionTokenRotation()
{
  def pin = this.source2();      // tainted
  def sessionToken = 100;        // clean: token corrente
  def THRESHOLD = 1000;

  while (pin < THRESHOLD)
  {
    sessionToken = 0;            // implicit flow
    pin = pin + 1;               // pin resta tainted (era già tainted)
  }

  pin = 500;                     // pin diventa clean (costante)

  // Simula un secondo tentativo con un nuovo PIN dal DB
  pin = this.source2();          // pin è di nuovo tainted

  while (pin < THRESHOLD)
  {
    sessionToken = sessionToken - 1; // implicit flow
    pin = pin + 1;
  }

  pin = 500;                     // clean di nuovo
  def backup = sessionToken;
  sessionToken = 999;

  this.sink2(sessionToken);
}
```

The cause could be that sessionToken is already Tainted in the abstract lattice due to the first loop, so the checker skips it because its logic only emits warnings on variables that were clean before the condition but after the first loop sessionToken is no longer clean.

Second Test

Abstract Domain: TaintThreeLevels

Checker: ImplicitFlow_Checker, TaintThreeLevelsChecker

Test File: ImplicitFlow_CheckerTest

Input Files: implicitflow_checker.imp

"warnings" : [{

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':114:17] on 'untyped hc::f4(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':116:24] on 'untyped hc::f4(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':116:24, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':135:21] on 'untyped hc::f5(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':137:20] on 'untyped hc::f5(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':137:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':147:20] on 'untyped hc::f5(hc* this)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':147:19"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':159:21] on 'untyped hc::f6(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':161:25] on 'untyped hc::f6(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':163:24] on 'untyped hc::f6(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':163:24, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':183:30] on 'untyped hc::f7(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':185:20] on 'untyped hc::f7(hc* this)': [EXPRESSION] Implicit flow detected at row
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':185:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':191:20] on 'untyped hc::f7(hc* this)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':191:19"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':24:13] on 'untyped hc::f0(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"
}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':25:20] on 'untyped hc::f0(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':25:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':43:16] on 'untyped hc::f1(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':45:20] on 'untyped hc::f1(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':45:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':51:16] on 'untyped hc::f1(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':53:18] on 'untyped hc::f1(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':53:18, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':61:20] on 'untyped hc::f1(hc* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':61:19"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':74:11] on 'untyped hc::f2(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':76:20] on 'untyped hc::f2(hc* this)': [EXPRESSION] Implicit flow detected at row 'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':76:20, variable modified inside a conditional statement, it should be considered tainted"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':93:21] on 'untyped hc::f3(hc* this)': [EXPRESSION] Tainted Condition, the implicit flow could affect variables in this branch"

}, {

"message" : "[inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':95:20] on 'untyped hc::f3(hc* this)': [EXPRESSION] Implicit flow detected at row

'inputs/homework/ImplicitFlow_Checker/implicitflow_checker.imp':95:20, variable modified inside a conditional statement, it should be considered tainted"

}]

Conclusions

After all the analysis performed and the test cases used, it is clear to say that there is a trade-off between soundness and precision in the static analysis. The soundness has always the priority on the precision leading to false positives (so every checker tend to issue more warnings that the actual ones). There was a pattern of behavior that the framework showed, the main imprecision sources came out multiple times and were easy recognizable: interprocedural context-insensitivity (so parameters initialized to Top), the widening in the loops (bounds to infinite) and the path-insensitivity on the conditional branches (in the if-else structure, the least upper bound merged states that were mutually exclusive on run-time). Moreover, the fact that a checker issued multiple warnings for compound expressions (like $[a+b]*c$) showed that the checkers think on the node of the control flow graph and not on the source code.

The fact that checkers tend to prioritize soundness over precision leading to false positive, assure that no real bug is lost and in the security field is preferred to the full precision.

Warnings Complete List (Checkers)

Div By Zero Checker Results

- 876957-div_by_zero.imp

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/876957-div_by_zero.imp]:128:33] on 'untyped  
RetryScheduler::computeShare(RetryScheduler* this, untyped total_work, untyped max_retries)':  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/876957-div_by_zero.imp]:157:28] on 'untyped  
SettlementEngine::splitAmount(SettlementEngine* this, untyped total, untyped participants)':  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/876957-div_by_zero.imp]:203:37] on 'untyped  
RequestDispatcher::computeShare(RequestDispatcher* this, untyped total_requests, untyped workers)':  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/876957-div_by_zero.imp]:238:26] on 'untyped  
AnalyticsPipeline::computeAverage(AnalyticsPipeline* this, untyped total, untyped count)':  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/876957-div_by_zero.imp]:273:26] on 'untyped  
HashTable::getShard(HashTable* this, untyped key, untyped table_size)': [EXPRESSION] This may be  
possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/876957-div_by_zero.imp]:87:32] on 'untyped  
MarsOrbiterNavigation::convertTrajectory(MarsOrbiterNavigation* this, untyped distance, untyped  
scale)': [EXPRESSION] This may be possible division by zero"  
} ]
```

- 881299_divbyzero.imp

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/881299_divbyzero.imp]:22:23] on 'untyped  
physics::compute_acceleration(physics* this, untyped force, untyped mass)': [EXPRESSION] This may  
be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/881299_divbyzero.imp]:26:26] on 'untyped  
physics::compute_speed(physics* this, untyped distance, untyped time)': [EXPRESSION] This may be  
possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/881299_divbyzero.imp]:30:24] on 'untyped  
physics::compute_energy_density(physics* this, untyped energy, untyped volume)': [EXPRESSION]  
This may be possible division by zero"  
} ]
```

- **896827-895879-divbyzero.imp**

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp]:13:33] on  
'untyped divbyzero_pentagon::correlatedSubtraction(divbyzero_pentagon* this, untyped x, untyped y)':  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp]:25:37] on  
'untyped divbyzero_pentagon::chainedUpperBounds(divbyzero_pentagon* this, untyped a, untyped b,  
untyped c)': [EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp]:38:37] on  
'untyped divbyzero_pentagon::remainderAsDivisor(divbyzero_pentagon* this, untyped x, untyped  
divisor)': [EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp]:60:47] on  
'untyped divbyzero_pentagon::privilegeLevelDivision(divbyzero_pentagon* this, untyped base_score,  
untyped penalty, untyped max_score)': [EXPRESSION] This may be possible division by zero"  
} ]
```

- **divbyzero.imp**

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:10:34] on 'untyped  
divbyzero::basic_1(divbyzero* this)': [EXPRESSION] This is definitely a division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:15:36] on 'untyped  
divbyzero::basic_2(divbyzero* this)': [EXPRESSION] This is definitely a division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:20:36] on 'untyped  
divbyzero::function_call_simple_compact(divbyzero* this)': [EXPRESSION] This may be possible  
division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:24:22] on 'untyped  
divbyzero::function_call_simple_expanded(divbyzero* this)': [EXPRESSION] This may be possible  
division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:37:20] on 'untyped  
divbyzero::function_call_branching(divbyzero* this)': [EXPRESSION] This may be possible division by  
zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:45:25] on 'untyped  
divbyzero::array_access(divbyzero* this)': [EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp]:52:26] on 'untyped  
divbyzero::looping(divbyzero* this)': [EXPRESSION] This may be possible division by zero"  
}, {
```

```

"message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp':59:34] on 'untyped
divbyzero::looping_indefinite(divbyzero* this, untyped p)': [EXPRESSION] This may be possible
division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp':67:38] on 'untyped
divbyzero::looping_indefinite_def_not_divbyzero(divbyzero* this, untyped p)': [EXPRESSION] This
may be possible division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/divbyzero.imp':89:16] on 'untyped
divbyzero::looping_array(divbyzero* this)': [EXPRESSION] This may be possible division by zero"
} ]

```

- **880119-890558-divbyzero.imp**

```

"warnings" : [ {
  "message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':18:22] on 'untyped
divbyzero::f1(divbyzero* this)': [EXPRESSION] This is definitely a division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':29:27] on 'untyped
divbyzero::f2(divbyzero* this)': [EXPRESSION] This may be possible division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':44:24] on 'untyped
divbyzero::f3(divbyzero* this)': [EXPRESSION] This is definitely a division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':61:22] on 'untyped
divbyzero::f4(divbyzero* this)': [EXPRESSION] This may be possible division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':80:23] on 'untyped
divbyzero::f5(divbyzero* this)': [EXPRESSION] This may be possible division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':88:26] on 'untyped
divbyzero::f6(divbyzero* this)': [EXPRESSION] This may be possible division by zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/880119-890558-divbyzero.imp':8:20] on 'untyped
divbyzero::f0(divbyzero* this)': [EXPRESSION] This is definitely a division by zero"
} ]

```

- **894069_div_by_zero.imp**

```

"warnings" : [ {
  "message" : "[inputs/programs_to_test/DivByZero/894069_div_by_zero.imp':21:33] on 'untyped
SensorGateway::computeAverage(SensorGateway* this)': [EXPRESSION] This is definitely a division by
zero"
}, {
"message" : "[inputs/programs_to_test/DivByZero/894069_div_by_zero.imp':32:27] on 'untyped
SensorGateway::computeRate(SensorGateway* this)': [EXPRESSION] This may be possible division by
zero"
} ]

```

- **894579_896954_divisionbyzero_1-2.imp**

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/894579_896954_divisionbyzero_1-2.imp':31:47] on  
'untyped FragmentEngine::computeFragments(FragmentEngine* this, untyped payloadSize):  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/894579_896954_divisionbyzero_1-2.imp':42:51] on  
'untyped FragmentEngine::computeFragmentsFixed(FragmentEngine* this, untyped payloadSize):  
[EXPRESSION] This may be possible division by zero" } ]
```

- **895227_897270_div_by_zero_interval_1.imp**

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/895227_897270_div_by_zero_interval_1.imp':12:41]  
on 'untyped ECommerce::applyDynamicDiscount(ECommerce* this)': [EXPRESSION] This may be  
possible division by zero"  
} ]
```

- **896827-895879-divbyzero_pentagon.imp**

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp':13:33] on  
'untyped divbyzero_pentagon::correlatedSubtraction(divbyzero_pentagon* this, untyped x, untyped y):  
[EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp':25:37] on  
'untyped divbyzero_pentagon::chainedUpperBounds(divbyzero_pentagon* this, untyped a, untyped b,  
untyped c)': [EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp':38:37] on  
'untyped divbyzero_pentagon::remainderAsDivisor(divbyzero_pentagon* this, untyped x, untyped  
divisor)': [EXPRESSION] This may be possible division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/896827-895879-divbyzero_pentagon.imp':60:47] on  
'untyped divbyzero_pentagon::privilegeLevelDivision(divbyzero_pentagon* this, untyped base_score,  
untyped penalty, untyped max_score)': [EXPRESSION] This may be possible division by zero"  
} ]
```

- **903942_div_by_zero.imp**

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/DivByZero/903942_div_by_zero.imp':33:31] on 'untyped  
div_by_zero::zero_divisor(div_by_zero* this)': [EXPRESSION] This is definitely a division by zero"  
}, {  
  "message" : "[inputs/programs_to_test/DivByZero/903942_div_by_zero.imp':55:30] on 'untyped  
div_by_zero::conditional_divisor(div_by_zero* this, untyped has_transactions)': [EXPRESSION] This  
may be possible division by zero"  
}, {
```

```
"message" : "[inputs/programs_to_test/DivByZero/903942_div_by_zero.imp':71:29] on 'untyped
div_by_zero::guarded_division(div_by_zero* this, untyped n)': [EXPRESSION] This may be possible
division by zero"
} ]
```

Taint Three Levels Checker Results

- 872966_TaintThreeLevels.imp

```
"warnings" : [ {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':104:37] on
'untyped SecurityGateway::testDataLeak(SecurityGateway* this)': [EXPRESSION] There is a possible
tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':104:32"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':78:43] on
'untyped SecurityGateway::processIncomingData(SecurityGateway* this, untyped priorityLevel,
untyped bypassShield)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':78:32"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':86:38] on
'untyped SecurityGateway::processIncomingData(SecurityGateway* this, untyped priorityLevel,
untyped bypassShield)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':86:32"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':90:38] on
'untyped SecurityGateway::processIncomingData(SecurityGateway* this, untyped priorityLevel,
untyped bypassShield)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/872966_TaintThreeLevels.imp':90:32"
} ]
```

- 880119-890558-taint-three-levels-2.imp

```
"warnings" : [ {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-
2.imp':26:38] on 'untyped shell::system_input_non_sanitized(shell* this, untyped req)': [EXPRESSION]
There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-
taint-three-levels-2.imp':26:33"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-
2.imp':34:37] on 'untyped shell::system_input_sanitized_completely(shell* this)': [EXPRESSION] There
is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-
three-levels-2.imp':34:33"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-
2.imp':43:37] on 'untyped shell::system_input_sanitized_conditional(shell* this, untyped req, untyped
q)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-2.imp':43:33"
} ]
```

```

}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-2.imp':58:36] on 'untyped shell::run_app(shell* this, untyped cmd, untyped flag)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-2.imp':58:32"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-2.imp':62:46] on 'untyped shell::run_app(shell* this, untyped cmd, untyped flag)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels-2.imp':62:45"
} ],

```

- 894004_taint.imp

```

"warnings" : [ {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':117:30] on 'untyped WebApplication::unsafeDirectQuery(WebApplication* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':117:24"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':129:30] on 'untyped WebApplication::sanitizedQuery(WebApplication* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':129:24"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':144:24] on 'untyped WebApplication::cookieInjection(WebApplication* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':144:19"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':155:28] on 'untyped WebApplication::sanitizedCookie(WebApplication* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':155:19"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':176:29] on 'untyped WebApplication::conditionalQuery(WebApplication* this, untyped isAdmin)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':176:24"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':193:29] on 'untyped WebApplication::conditionalSanitized(WebApplication* this, untyped isAdmin)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':193:24"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':213:29] on 'untyped WebApplication::multiStepInjection(WebApplication* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':213:24"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':227:29] on 'untyped WebApplication::multiStepSanitized(WebApplication* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':227:24"
}, {

```

```
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':250:29] on 'untyped
WebApplication::loopInjection(WebApplication* this, untyped n)': [EXPRESSION] There is a possible
tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894004_taint.imp':250:24"
} ]
```

- **1003406_three_taint_1.imp**

```
"warnings" : [ {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':119:37] on
'untyped CommandInjectionScenario::runReportConditionalSanitize(CommandInjectionScenario* this,
untyped isAdminMode)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':119:34"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':140:37] on
'untyped CommandInjectionScenario::runReportAfterReTaint(CommandInjectionScenario* this)':
[EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':140:34"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':171:37] on
'untyped CommandInjectionScenario::runViaHelper(CommandInjectionScenario* this)':
[EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':171:34"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':191:37] on
'untyped CommandInjectionScenario::runViaHelperSafe(CommandInjectionScenario* this)':
[EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':191:34"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':211:41] on
'untyped CommandInjectionScenario::batchProcessUnsafe(CommandInjectionScenario* this, untyped
numJobs)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':211:38"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':229:45] on
'untyped CommandInjectionScenario::batchProcessSafe(CommandInjectionScenario* this, untyped
numJobs)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':229:38"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':254:37] on
'untyped CommandInjectionScenario::runWithTwoSources(CommandInjectionScenario* this)':
[EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':254:34"
}, {
  "message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':78:37] on
'untyped CommandInjectionScenario::runReportUnsafe(CommandInjectionScenario* this)':
[EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':78:34"
}, {
```

```
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':98:37] on 'untyped CommandInjectionScenario::runReportSafe(CommandInjectionScenario* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/1003406_three_taint_1.imp':98:34"
} ]
```

- **876957-taint.imp**

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/876957-taint.imp':76:29] on 'untyped SqlInjection::testDirectInjection(SqlInjection* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/876957-taint.imp':76:24"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/876957-taint.imp':82:28] on 'untyped SqlInjection::testSanitizedQuery(SqlInjection* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/876957-taint.imp':82:24"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/876957-taint.imp':93:29] on 'untyped SqlInjection::testMixedQuery(SqlInjection* this, untyped isAdmin)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/876957-taint.imp':93:24"
} ]
```

- **880119-890558-taint-three-levels.imp**

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels.imp':25:14] on 'untyped taintthreelevels::basic2(taintthreelevels* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels.imp':25:13"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels.imp':33:14] on 'untyped taintthreelevels::basic3(taintthreelevels* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels.imp':33:13"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels.imp':45:14] on 'untyped taintthreelevels::branch(taintthreelevels* this, untyped b)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/880119-890558-taint-three-levels.imp':45:13"
} ]
```

- **881299_taint.imp**

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':47:30] on 'untyped webapi::test1(webapi* this)': [EXPRESSION] There is a possible tainted value in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':47:24"
}, {
```

```

"message" : "[inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':54:34] on 'untyped
webapi::test2(webapi* this)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':54:24"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':74:32] on 'untyped
webapi::test5(webapi* this)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':74:24"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':82:33] on 'untyped
webapi::test6(webapi* this)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/881299_taint.imp':82:24"
} ]

```

- **894069_taint.imp**

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/894069_taint.imp':43:29] on 'untyped
MedicalPortal::lookupPatient(MedicalPortal* this)': [EXPRESSION] There is a possible tainted value in
a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894069_taint.imp':43:24"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/894069_taint.imp':49:28] on 'untyped
MedicalPortal::storePatientSafe(MedicalPortal* this)': [EXPRESSION] There is a possible tainted value
in a sink: 'inputs/programs_to_test/ThreeLevelsTaint/894069_taint.imp':49:24"
} ]

```

- **894579_896954_taintthreelevel_1-2.imp**

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/894579_896954_taintthreelevel_1-
2.imp':102:60] on 'untyped JavaServerFlawed::processFileRequest(JavaServerFlawed* this, untyped
request)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/894579_896954_taintthreelevel_1-2.imp':102:50"
}, {
"message" : "[inputs/programs_to_test/ThreeLevelsTaint/894579_896954_taintthreelevel_1-
2.imp':129:59] on 'untyped JavaServerSecure::processFileRequest(JavaServerSecure* this, untyped
request)': [EXPRESSION] There is a possible tainted value in a sink:
'inputs/programs_to_test/ThreeLevelsTaint/894579_896954_taintthreelevel_1-2.imp':129:50"
} ]

```

- **895227_897270_taint_three_level_1.imp**

```

"warnings" : []

```

String Checker Results

- 872966_Strings.imp

HTTPStringChecker

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':119:43] on 'untyped  
WebApiClient::notifyDeployPipeline(WebApiClient* this, untyped buildId)': [EXPRESSION] The function  
uses an http/https url in the following parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':134:43] on 'untyped  
WebApiClient::pollHealthEndpoints(WebApiClient* this)': [EXPRESSION] The function uses an http/https  
url in the following parameters: 2nd"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':262:30] on 'untyped  
WebApiClient::sendTransactionalEmail(WebApiClient* this)': [EXPRESSION] The function uses an  
http/https url in the following parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':272:30] on 'untyped  
WebApiClient::downloadPresignedAsset(WebApiClient* this)': [EXPRESSION] The function uses an  
http/https url in the following parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':44:30] on 'untyped  
WebApiClient::fetchMetricsDashboard(WebApiClient* this)': [EXPRESSION] The function uses an  
http/https url in the following parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':54:40] on 'untyped  
WebApiClient::refreshOAuthToken(WebApiClient* this)': [EXPRESSION] The function uses an http/https  
url in the following parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':95:35] on 'untyped  
WebApiClient::resolveWithFailover(WebApiClient* this, untyped usePrimary)': [EXPRESSION] The  
function uses an http/https url in the following parameters: 1st"  
} ]
```

DotComStringChecker

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':148:27] on 'untyped  
WebApiClient::connectPaymentGateway(WebApiClient* this)': [EXPRESSION] The function uses a  
.com in the following parameters: 1st"
```

```

}, {
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':236:32] on 'untyped WebApiClient::routeToActiveRegion(WebApiClient* this, untyped isPrimary)': [EXPRESSION] The function uses a .com in the following parameters: 1st"
}, {
  "message" : "[inputs/programs_to_test/SuffixPrefix/872966_Strings.imp':262:30] on 'untyped WebApiClient::sendTransactionalEmail(WebApiClient* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"
} ]

```

- **880119-890558-dotcom-strings.imp**

HTTPStringChecker

"warnings" : []

DotComStringChecker

"warnings" : [{

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':113:32] on 'untyped DotComStrings::cdnRouter(DotComStrings* this, untyped region)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':136:53] on 'untyped DotComStrings::phishingEmailDetection(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':172:37] on 'untyped DotComStrings::dnsRebindingSetup(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':24:31] on 'untyped DotComStrings::directDotCom(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':32:41] on 'untyped DotComStrings::directDotComEmail(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':54:43] on 'untyped DotComStrings::buildEmailAddress(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':70:47] on 'untyped DotComStrings::corporateEmailSystem(DotComStrings* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-dotcom-strings.imp':92:41] on 'untyped DotComStrings::serverSelection(DotComStrings* this, untyped useBackup)': [EXPRESSION] The function uses a .com in the following parameters: 1st"

}]

- **894579_896954_prefixsuffix_5.imp**

[HTTPStringChecker](#)

"warnings" : [{

"message" : "[inputs/programs_to_test/SuffixPrefix/894579_896954_prefixsuffix_5.imp':19:40] on 'untyped ProtocolUpgrader::testUpgrade(ProtocolUpgrader* this)': [EXPRESSION] The function uses an http/https url in the following parameters: 1st"}]

[DotComStringChecker](#)

"warnings" : [{

"message" : "[inputs/programs_to_test/SuffixPrefix/894579_896954_prefixsuffix_5.imp':19:40] on 'untyped ProtocolUpgrader::testUpgrade(ProtocolUpgrader* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"}],

- **896827-895879-prefix.imp**

[HTTPStringChecker](#)

"warnings" : [{

"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':14:22] on 'untyped prefix::insecureUrl(prefix* this)': [EXPRESSION] The function uses an http/https url in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':20:22] on 'untyped prefix::secureUrl(prefix* this)': [EXPRESSION] The function uses an http/https url in the following parameters: 1st"

}, {

"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':30:28] on 'untyped prefix::joinedHttpUrls(prefix* this, untyped flag)': [EXPRESSION] The function uses an http/https url in the following parameters: 1st"

}, {

```
"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':47:36] on 'untyped
prefix::multipleCalls(prefix* this, untyped flag)': [EXPRESSION] The function uses an http/https url in
the following parameters: 1st"
```

```
}, {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':89:67] on 'untyped
prefix::generateReportVulnerable(prefix* this)': [EXPRESSION] The function uses an http/https url in
the following parameters: 1st"
```

```
} ],
```

DotComStringChecker

```
"warnings" : []
```

- **876957-strings.imp**

HTTPStringChecker

```
"warnings" : [ {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':126:38] on 'untyped
Redirector::testHttpsDotCom(Redirector* this)': [EXPRESSION] The function uses an http/https url in
the following parameters: 1st"
```

```
}, {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':154:38] on 'untyped
UrlBuilder::buildAndCall(UrlBuilder* this, untyped host)': [EXPRESSION] The function uses an
http/https url in the following parameters: 1st"
```

```
}, {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':51:35] on 'untyped
WebScraper::testHttpsUrl(WebScraper* this)': [EXPRESSION] The function uses an http/https url in the
following parameters: 1st"
```

```
}, {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':57:36] on 'untyped
WebScraper::testHttpUrl(WebScraper* this)': [EXPRESSION] The function uses an http/https url in the
following parameters: 1st"
```

```
} ]
```

DotComStringChecker

```
"warnings" : [ {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':126:38] on 'untyped
Redirector::testHttpsDotCom(Redirector* this)': [EXPRESSION] The function uses a .com in the
following parameters: 1st"
```

```
}, {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':181:39] on 'untyped
DomainBuilder::buildAndLookup(DomainBuilder* this, untyped brand)': [EXPRESSION] The function
uses a .com in the following parameters: 1st"
```

```
}, {
```

```
"message" : "[inputs/programs_to_test/SuffixPrefix/876957-strings.imp':89:40] on 'untyped  
DnsResolver::testDotComDomain(DnsResolver* this)': [EXPRESSION] The function uses a .com in the  
following parameters: 1st"
```

```
} ],
```

- **880119-890558-http-strings.imp**

HTTPStringChecker

```
"warnings" : [ {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':116:34] on  
'untyped HTTPStrings::regionRouter(HTTPStrings* this, untyped regionCode)': [EXPRESSION] The  
function uses an http/https url in the following parameters: 1st"  
, {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':142:35] on  
'untyped HTTPStrings::proxyServiceSSRF(HTTPStrings* this)': [EXPRESSION] The function uses an  
http/https url in the following parameters: 1st"  
, {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':24:22] on 'untyped  
HTTPStrings::directHTTP(HTTPStrings* this)': [EXPRESSION] The function uses an http/https url in  
the following parameters: 1st"  
, {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':31:22] on 'untyped  
HTTPStrings::directHTTPS(HTTPStrings* this)': [EXPRESSION] The function uses an http/https url in  
the following parameters: 1st"  
, {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':54:37] on 'untyped  
HTTPStrings::apiClientConstruction(HTTPStrings* this)': [EXPRESSION] The function uses an  
http/https url in the following parameters: 1st"  
, {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':71:40] on 'untyped  
HTTPStrings::webhookRegistration(HTTPStrings* this)': [EXPRESSION] The function uses an  
http/https url in the following parameters: 1st"  
, {  
"message" : "[inputs/programs_to_test/SuffixPrefix/880119-890558-http-strings.imp':94:35] on 'untyped  
HTTPStrings::loadBalancer(HTTPStrings* this, untyped serverLoad)': [EXPRESSION] The function  
uses an http/https url in the following parameters: 1st"  
} ]
```

DotComStringChecker

```
"warnings" : []
```

- **881299_presuffix.imp**

HTTPStringChecker

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/881299_presuffix.imp':42:66] on 'untyped  
router::main(router* this)': [EXPRESSION] The function uses an http/https url in the following  
parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/881299_presuffix.imp':43:56] on 'untyped  
router::main(router* this)': [EXPRESSION] The function uses an http/https url in the following  
parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/881299_presuffix.imp':45:34] on 'untyped  
router::main(router* this)': [EXPRESSION] The function uses an http/https url in the following  
parameters: 1st"  
} ]
```

DotComStringChecker

```
"warnings" : [ {  
  
  "message" : "[inputs/programs_to_test/SuffixPrefix/881299_presuffix.imp':42:66] on 'untyped  
router::main(router* this)': [EXPRESSION] The function uses a .com in the following parameters: 1st"  
  
} ]
```

- **894069_prefix_suffix.imp**

HTTPStringChecker

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/894069_prefix_suffix.imp':21:40] on 'untyped  
FileManager::uploadToCloud(FileManager* this)': [EXPRESSION] The function uses an http/https url  
in the following parameters: 1st"  
}, {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/894069_prefix_suffix.imp':41:47] on 'untyped  
FileManager::fetchRemoteLog(FileManager* this)': [EXPRESSION] The function uses an http/https url  
in the following parameters: 1st"  
} ]
```

DotComStringChecker

```
"warnings" : []
```

- **895227_897270_dot_com_string_3.imp**

HTTPStringChecker

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':14:22] on 'untyped  
prefix::insecureUrl(prefix* this)': [EXPRESSION] The function uses an http/https url in the following  
parameters: 1st"  
}, {
```

```

"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':20:22] on 'untyped
prefix::secureUrl(prefix* this)': [EXPRESSION] The function uses an http/https url in the following
parameters: 1st"
}, {
"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':30:28] on 'untyped
prefix::joinedHttpUrls(prefix* this, untyped flag)': [EXPRESSION] The function uses an http/https url in
the following parameters: 1st"
}, {
"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':47:36] on 'untyped
prefix::multipleCalls(prefix* this, untyped flag)': [EXPRESSION] The function uses an http/https url in
the following parameters: 1st"
}, {
"message" : "[inputs/programs_to_test/SuffixPrefix/896827-895879-prefix.imp':89:67] on 'untyped
prefix::generateReportVulnerable(prefix* this)': [EXPRESSION] The function uses an http/https url in
the following parameters: 1st"
} ]

```

DotComStringChecker

```
"warnings" : []
```

- 895227_897270_http_string_3.imp

HTTPStringChecker

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/SuffixPrefix/895227_897270_http_string_3.imp':9:41] on
'untyped WebhookNotifier::sendNotification(WebhookNotifier* this)': [EXPRESSION] The function
uses an http/https url in the following parameters: 1st"
} ]

```

DotComStringChecker

```
"warnings" : []
```

Overflow/Underflow Checker Results

- 876957-overflow.imp

OVERFLOW 8-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:103:25] on 'untyped  
overflow::testFloatUnderflow8Bit(overflow* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:110:25] on 'untyped  
overflow::testFloatOverflow16Bit(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:155:42] on 'untyped  
bankAccount::withdraw(bankAccount* this, untyped amount)': [EXPRESSION] This is an  
over/underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:21:31] on 'untyped  
overflow::test8BitOverflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:27:31] on 'untyped  
overflow::test8BitUnderflow(overflow* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:44:32] on 'untyped  
overflow::test16BitOverflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:50:32] on 'untyped  
overflow::test16BitUnderflow(overflow* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:60:30] on 'untyped  
overflow::test32BitOverflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:66:30] on 'untyped  
overflow::test32BitUnderflow(overflow* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:77:25] on 'untyped  
overflow::testMultiplicationOverflow8Bit(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:84:25] on 'untyped  
overflow::testMultiplicationSafe32Bit(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:96:25] on 'untyped  
overflow::testFloatOverflow8Bit(overflow* this)': [EXPRESSION] This is an overflow"  
} ],
```

OVERFLOW 16-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp]:110:25] on 'untyped  
overflow::testFloatOverflow16Bit(overflow* this)': [EXPRESSION] This is an overflow"  
}, {
```

```

"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':147:42] on 'untyped
bankAccount::deposit(bankAccount* this, untyped amount)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':155:42] on 'untyped
bankAccount::withdraw(bankAccount* this, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':155:42] on 'untyped
bankAccount::withdraw(bankAccount* this, untyped amount)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':44:32] on 'untyped
overflow::test16BitOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':50:32] on 'untyped
overflow::test16BitUnderflow(overflow* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':60:30] on 'untyped
overflow::test32BitOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':66:30] on 'untyped
overflow::test32BitUnderflow(overflow* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':84:25] on 'untyped
overflow::testMultiplicationSafe32Bit(overflow* this)': [EXPRESSION] This is an overflow"
} ]

```

OVERFLOW 32BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':147:42] on 'untyped
bankAccount::deposit(bankAccount* this, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':147:42] on 'untyped
bankAccount::deposit(bankAccount* this, untyped amount)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':155:42] on 'untyped
bankAccount::withdraw(bankAccount* this, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':155:42] on 'untyped
bankAccount::withdraw(bankAccount* this, untyped amount)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/876957-overflow.imp':60:30] on 'untyped
overflow::test32BitOverflow(overflow* this)': [EXPRESSION] This is an overflow"
} ]

```

- **880119-890558-overflow.imp**

OVERFLOW 8-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':15:20] on  
'untyped overflow::f1(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':27:29] on  
'untyped overflow::f2(overflow* this)': [EXPRESSION] This is an over/underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':35:20] on  
'untyped overflow::f3(overflow* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':46:25] on  
'untyped overflow::f4(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':50:25] on  
'untyped overflow::f4(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':59:20] on  
'untyped overflow::f5(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':6:20] on  
'untyped overflow::f0(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':74:25] on  
'untyped overflow::f5(overflow* this)': [EXPRESSION] This is an over/underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':7:20] on  
'untyped overflow::f0(overflow* this)': [EXPRESSION] This is an overflow"  
} ]
```

OVERFLOW 16-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':15:20] on  
'untyped overflow::f1(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':27:29] on  
'untyped overflow::f2(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':35:20] on  
'untyped overflow::f3(overflow* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':46:25] on  
'untyped overflow::f4(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':59:20] on  
'untyped overflow::f5(overflow* this)': [EXPRESSION] This is an overflow"
```

```

}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':6:20] on
'untyped overflow::f0(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':74:25] on
'untyped overflow::f5(overflow* this)': [EXPRESSION] This is an over/underflow"
} ]

```

OVERFLOW 32BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':15:20] on
'untyped overflow::f1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':35:20] on
'untyped overflow::f3(overflow* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':46:25] on
'untyped overflow::f4(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':59:20] on
'untyped overflow::f5(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':6:20] on
'untyped overflow::f0(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/880119-890558-overflow.imp':74:25] on
'untyped overflow::f5(overflow* this)': [EXPRESSION] This is an over/underflow"
} ]

```

- 894579_896954_overunderflow_1-2.imp

OVERFLOW 8-BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894579_896954_overunderflow_1-
2.imp':20:35] on 'untyped FsContext::appendParam(FsContext* this, untyped data, untyped len)':
[EXPRESSION] This is an over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894579_896954_overunderflow_1-
2.imp':20:44] on 'untyped FsContext::appendParam(FsContext* this, untyped data, untyped len)':
[EXPRESSION] This is an over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894579_896954_overunderflow_1-
2.imp':30:43] on 'untyped FsContext::appendParamFixed(FsContext* this, untyped data, untyped len)':
[EXPRESSION] This is an over/underflow"
}, {

```



```
"message" : "[inputs/programs_to_test/Overflow/Underflow/894579_896954_overunderflow_1-2.imp':35:45] on 'untyped FsContext::appendParamFixed(FsContext* this, untyped data, untyped len)': [EXPRESSION] This is an over/underflow"
} ]
```

- **overflow8.imp**

OVERFLOW 8-BITS

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':15:38] on 'untyped overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':17:30] on 'untyped overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':20:30] on 'untyped overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':29:38] on 'untyped overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':31:30] on 'untyped overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':34:30] on 'untyped overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':43:29] on 'untyped overflow::byte_array_access(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':56:36] on 'untyped overflow::byte_function_call(overflow* this)': [EXPRESSION] This is an over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':5:22] on 'untyped overflow::byte_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':7:22] on 'untyped overflow::byte_1(overflow* this)': [EXPRESSION] This is an underflow"
} ]
```

OVERFLOW 16-BITS

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':15:38] on 'untyped overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':17:30] on 'untyped overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}
```

```

}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':20:30] on 'untyped
overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':29:38] on 'untyped
overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':31:30] on 'untyped
overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':34:30] on 'untyped
overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
} ],

```

OVERFLOW 32BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':15:38] on 'untyped
overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':17:30] on 'untyped
overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':20:30] on 'untyped
overflow::byte_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':29:38] on 'untyped
overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':31:30] on 'untyped
overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow8.imp':34:30] on 'untyped
overflow::byte_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
} ]

```

- 881299_overunderflow.imp

OVERFLOW 8-BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':22:25] on
'untyped bank::deposit(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':26:25] on
'untyped bank::withdraw(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {

```

```
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':30:25] on
'untyped bank::apply_interest(bank* this, untyped balance, untyped rate)': [EXPRESSION] This is an
over/underflow"
} ],
```

OVERFLOW 16-BITS

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':22:25] on
'untyped bank::deposit(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':22:25] on
'untyped bank::deposit(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':26:25] on
'untyped bank::withdraw(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':26:25] on
'untyped bank::withdraw(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':30:25] on
'untyped bank::apply_interest(bank* this, untyped balance, untyped rate)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':30:25] on
'untyped bank::apply_interest(bank* this, untyped balance, untyped rate)': [EXPRESSION] This is an
overflow"
} ]
```

OVERFLOW 32BITS

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':22:25] on
'untyped bank::deposit(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':22:25] on
'untyped bank::deposit(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overunderflow.imp':26:25] on
'untyped bank::withdraw(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
over/underflow"
}, {
```

```

"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overflow.imp':26:25] on
'untyped bank::withdraw(bank* this, untyped balance, untyped amount)': [EXPRESSION] This is an
overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overflow.imp':30:25] on
'untyped bank::apply_interest(bank* this, untyped balance, untyped rate)': [EXPRESSION] This is an
over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/881299_overflow.imp':30:25] on
'untyped bank::apply_interest(bank* this, untyped balance, untyped rate)': [EXPRESSION] This is an
overflow"
} ]

```

- 894069_overflow.imp

OVERFLOW 8-BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':21:29] on 'untyped
GameEngine::addBonusScore(GameEngine* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':28:26] on 'untyped
GameEngine::applyDamage(GameEngine* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':37:34] on 'untyped
GameEngine::simulateVelocity(GameEngine* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':38:30] on 'untyped
GameEngine::simulateVelocity(GameEngine* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':47:29] on 'untyped
GameEngine::levelUpScore(GameEngine* this)': [EXPRESSION] This is an overflow"
} ]

```

OVERFLOW 16-BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':21:29] on 'untyped
GameEngine::addBonusScore(GameEngine* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':28:26] on 'untyped
GameEngine::applyDamage(GameEngine* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':37:34] on 'untyped
GameEngine::simulateVelocity(GameEngine* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':38:30] on 'untyped
GameEngine::simulateVelocity(GameEngine* this)': [EXPRESSION] This is an underflow"
} ]

```

```
} ]
```

OVERFLOW 32BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':21:29] on 'untyped  
GameEngine::addBonusScore(GameEngine* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':28:26] on 'untyped  
GameEngine::applyDamage(GameEngine* this)': [EXPRESSION] This is an underflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':37:34] on 'untyped  
GameEngine::simulateVelocity(GameEngine* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/894069_overflow.imp':38:30] on 'untyped  
GameEngine::simulateVelocity(GameEngine* this)': [EXPRESSION] This is an underflow"  
} ]
```

- 895227_897270_overflow_interval_1.imp

OVERFLOW 8-BITS

```
"warnings" : [ {  
  "message" :  
  "[inputs/programs_to_test/Overflow/Underflow/895227_897270_overflow_interval_1.imp':11:34] on  
'untyped CryptoMiner::calculateHashRate(CryptoMiner* this, untyped inputA, untyped inputB)':  
[EXPRESSION] This is an overflow"  
} ]
```

OVERFLOW 16-BITS

```
"warnings" : [ {  
  "message" :  
  "[inputs/programs_to_test/Overflow/Underflow/895227_897270_overflow_interval_1.imp':11:34] on  
'untyped CryptoMiner::calculateHashRate(CryptoMiner* this, untyped inputA, untyped inputB)':  
[EXPRESSION] This is an overflow"  
} ]
```

OVERFLOW 32BITS

```
"warnings" : [ {  
  "message" :  
  "[inputs/programs_to_test/Overflow/Underflow/895227_897270_overflow_interval_1.imp':11:34] on  
'untyped CryptoMiner::calculateHashRate(CryptoMiner* this, untyped inputA, untyped inputB)':  
[EXPRESSION] This is an overflow"  
} ]
```

- 896827-895879-overflow.imp

OVERFLOW 8-BITS

```

"warnings" : [ {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':10:20] on
'untyped overflow::bankAccountOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':19:22] on
'untyped overflow::temperatureUnderflow(overflow* this)': [EXPRESSION] This is an underflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':28:32] on
'untyped overflow::safeBoundedCounter(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':30:30] on
'untyped overflow::safeBoundedCounter(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':38:25] on
'untyped overflow::constantOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':70:31] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':75:39] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':77:28] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':9:32] on
'untyped overflow::bankAccountOverflow(overflow* this)': [EXPRESSION] This is an over/underflow"
} ]

```

OVERFLOW 16-BITS

```

"warnings" : [ {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':10:20] on
'untyped overflow::bankAccountOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':28:32] on
'untyped overflow::safeBoundedCounter(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':30:30] on
'untyped overflow::safeBoundedCounter(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':38:25] on
'untyped overflow::constantOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':70:31] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {

```

```
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':75:39] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':77:28] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':9:32] on
'untyped overflow::bankAccountOverflow(overflow* this)': [EXPRESSION] This is an over/underflow"
} ]
```

OVERFLOW 32BITS

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':10:20] on
'untyped overflow::bankAccountOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':28:32] on
'untyped overflow::safeBoundedCounter(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':30:30] on
'untyped overflow::safeBoundedCounter(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':70:31] on
'untyped overflow::cartPriceOverflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/896827-895879-overflow.imp':9:32] on
'untyped overflow::bankAccountOverflow(overflow* this)': [EXPRESSION] This is an over/underflow"
} ]
```

- 903942_overflow.imp

OVERFLOW 8-BITS

```
"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':34:22] on 'untyped
overflow::byte_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':48:23] on 'untyped
overflow::byte_underflow(overflow* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':63:28] on 'untyped
overflow::short_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':84:24] on 'untyped
overflow::loop_accumulation(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':85:20] on 'untyped
overflow::loop_accumulation(overflow* this)': [EXPRESSION] This is an overflow"
} ]
```

- OVERFLOW 16-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':63:28] on 'untyped  
overflow::short_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':84:24] on 'untyped  
overflow::loop_accumulation(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':85:20] on 'untyped  
overflow::loop_accumulation(overflow* this)': [EXPRESSION] This is an overflow"  
} ]
```

OVERFLOW 32BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':84:24] on 'untyped  
overflow::loop_accumulation(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/903942_overflow.imp':85:20] on 'untyped  
overflow::loop_accumulation(overflow* this)': [EXPRESSION] This is an overflow"  
} ]
```

- overflow16.imp

OVERFLOW 8-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':16:38] on 'untyped  
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':18:32] on 'untyped  
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':21:32] on 'untyped  
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':30:38] on 'untyped  
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':32:32] on 'untyped  
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':35:32] on 'untyped  
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':44:31] on 'untyped  
overflow::short_array_access(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':55:37] on 'untyped  
overflow::short_function_call(overflow* this)': [EXPRESSION] This is an over/underflow"
```

```

}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':57:38] on 'untyped
overflow::short_function_call(overflow* this)': [EXPRESSION] This is an over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':6:24] on 'untyped
overflow::short_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':8:24] on 'untyped
overflow::short_1(overflow* this)': [EXPRESSION] This is an underflow"
} ]

```

OVERFLOW 16-BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':16:38] on 'untyped
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':18:32] on 'untyped
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':21:32] on 'untyped
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':30:38] on 'untyped
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':32:32] on 'untyped
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':35:32] on 'untyped
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':44:31] on 'untyped
overflow::short_array_access(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':55:37] on 'untyped
overflow::short_function_call(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':6:24] on 'untyped
overflow::short_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':8:24] on 'untyped
overflow::short_1(overflow* this)': [EXPRESSION] This is an underflow"
} ]

```

OVERFLOW 32BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':16:38] on 'untyped  
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':18:32] on 'untyped  
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':21:32] on 'untyped  
overflow::short_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':30:38] on 'untyped  
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':32:32] on 'untyped  
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow16.imp':35:32] on 'untyped  
overflow::short_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
} ]
```

- overflow32.imp

OVERFLOW 8-BITS

```
"warnings" : [ {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':14:35] on 'untyped  
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':16:38] on 'untyped  
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':18:36] on 'untyped  
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':21:36] on 'untyped  
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':28:35] on 'untyped  
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':30:38] on 'untyped  
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':32:36] on 'untyped  
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {  
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':35:36] on 'untyped  
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"  
}, {
```

```

"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':44:35] on 'untyped
overflow::integer_array_access(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':55:41] on 'untyped
overflow::integer_function_call(overflow* this)': [EXPRESSION] This is an over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':57:42] on 'untyped
overflow::integer_function_call(overflow* this)': [EXPRESSION] This is an over/underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':6:28] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':6:41] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':8:28] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an underflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':8:41] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an underflow"
} ]

```

OVERFLOW 16-BITS

```

"warnings" : [ {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':14:35] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':16:38] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':18:36] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':21:36] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':28:35] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':30:38] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':32:36] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':35:36] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
} ]

```

```

}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':44:35] on 'untyped
overflow::integer_array_access(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':55:41] on 'untyped
overflow::integer_function_call(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':6:28] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':6:41] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':8:28] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an underflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':8:41] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an underflow"
} ]

```

OVERFLOW 32BITS

```

"warnings" : [ {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':16:38] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':18:36] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':21:36] on 'untyped
overflow::integer_loop_branching_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':30:38] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':32:36] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':35:36] on 'untyped
overflow::integer_loop_branching_no_overflow(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':44:35] on 'untyped
overflow::integer_array_access(overflow* this)': [EXPRESSION] This is an overflow"
}, {
  "message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':55:41] on 'untyped
overflow::integer_function_call(overflow* this)': [EXPRESSION] This is an overflow"
}, {

```

```
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':6:41] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an overflow"
}, {
"message" : "[inputs/programs_to_test/Overflow/Underflow/overflow32.imp':8:41] on 'untyped
overflow::integer_1(overflow* this)': [EXPRESSION] This is an underflow"
} ]
```